

Etude d'un algorithme de vérification de modèle en Logique Linéaire Temporelle

Dans un monde dont la dépendance à divers technologies est très importante, il est important de s'assurer du bon fonctionnement de ces technologies. C'est ainsi que la vérification de modèle rentre en jeu pour répondre à ces problématiques. Ce sont des techniques automatiques qui étant donné un modèle à nombre d'état fini d'un système et des propriétés formelles, vérifie systématiquement si ces propriétés restent vrai pour ce modèle. Nous allons donc étudier un algorithme proposé par Rob Gerth, Doron Peled, Moshe Ya'akov Vardi, Pierre Wolper en 1995.

La bible: Simple On-the-fly Automatic Verification of Linear Temporal Logic

1 Préliminaires

Nous allons pour l'instant introduire des concepts hors programme MPI afin de mener à bien notre étude.

Pour illustrer les différentes définitions et leurs utilités, on considerera un feu de circulation.

1.1 Logique Linéaire Temporelle

On modélise les différentes contraintes auxquels notre système doivent obéir par des formules logiques "dépendant du temps" d'où:

Definition 1 (Formule de la LTL): On définit les opérateurs suivants:

- $X\phi$: ϕ doit être satisfaite dans l'état suivant (neXt)
- $\psi U \phi$: ψ doit être satisfaite dans tous les états jusqu'à un état où ϕ est satisfait (Until)

On définit F l'ensemble des formules de la LTL inductivement par:

- $p \in AP \implies p \in F$
- si ψ et ϕ sont des formules de LTL alors $\neg\psi, \phi \vee \psi, X\psi, \phi U \psi$ sont des formules de LTL

AP un ensemble fini de variables propositionnelles.

On introduit aussi $\psi V \phi = \neg(\neg\psi U \neg\phi)$.

Soit $v : \omega \times F \rightarrow \{T, F\}$ une fonction de valuation.

Definition 2 (Monde): On définit un tel objet comme $\omega := w_0, w_1, \dots$ une suite infinie d'état. On écrira $\omega^i := w_i, w_{i+1}, \dots$ un suffixe de ω . On définit $\omega \models f$ via:

- $\omega \models a \iff v(w_0, a) = T$ si a atomique
- $\omega \models \neg f$ si $\neg(\omega \models f)$
- $\omega \models f \vee g$ si $\omega \models f$ ou $\omega \models g$
- $\omega \models Xf$ si $\omega^1 \models f$
- $\omega \models f U g$ si $\exists i \geq 0 | \omega_i \models g$ et $\forall j \in [0, i-1], \omega_j \models f$

1.2 Automate de Buchi

Ces formules seront transformés en automate acceptant des mots infinis, donc on doit définir de nouveaux automates:

Definition 3 (Automate de Buchi (BA)): Un automate de Büchi est un 5-uplets (S, I, T, F, Σ) tel que

- S est un ensemble fini d'état
- $I \subseteq S$ un ensemble d'état initiaux
- $F \subseteq S$ un ensemble d'état finaux
- Σ Un ensemble fini de symboles appelé alphabet
- $T : \{S \times \Sigma\} \mapsto \mathcal{P}(S)$ Une fonction de transition.

Cet automate particulier accepte des séquences infinis (donc des mots infinis) ssi le mot passe par un état acceptant une infinité de fois.

On doit pouvoir faire deux opérations sur nos automates, un test pour savoir si un automate est vide et intersecter des automates, tout en intersectant les langages naturellement.

Proposition 1 (Test BA Vide): Un BA est non vide ssi il existe un état final qui est atteignable depuis un état initial et appartient à un cycle.

Preuve 1: $(\Leftrightarrow)$ L'équivalence est evidente, c'est une reformulation

On peut utiliser l'algorithme de Tarjan par exemple pour trouver les composantes fortement connexe (ou Kosaraju Sharir)

Algorithm 1 Cycle Detection

Require: $G = (S, A), A = (S, I, T, F, \Sigma)$

Ensure: A is empty

```

 $C_1, \dots, C_k \leftarrow Tarjan(G)$ 
 $f_1, \dots, f_n \leftarrow F$ 
 $i_1, \dots, i_p \leftarrow I$ 
for  $i$  from 1 to  $n$  do
   $C \leftarrow C_q$  such that  $f_i \in C_q$ 
  if  $|C| > 1$  then
    for  $j$  from 1 to  $p$  do
       $L \leftarrow DFS(G, i_j)$ 
      if  $f_i \in L$  then return True
    end if
  end for
end if
end for
return False

```

Notre algorithme à un moment un automate de Büchi généralisé, on va devoir donc le transformer en automate de Büchi usuel.

Definition 4 (Automate de Büchi généralisé (GBA)): C'est un 4-uplets $(S, I, \rightarrow, F_g)$ tel que:

- S est un ensemble fini d'état
- $I \subseteq S$ un ensemble d'état initiaux
- $\rightarrow \subseteq S \times S$ est la relation de transition
- $F_g \subseteq \mathcal{P}(S)$ est un ensemble d'ensemble d'état acceptant

Nous n'avons pas encore proprement défini ici la condition d'acceptation pour un GBA, il nous faut déjà étiqueté notre automate pour cela

Definition 5 (Automate de Büchi généralisé étiqueté (LGBA)): C'est un triplet (A, Σ, L) où:

- A est un GBA
- Σ un alphabet/domaine finie
- $L : S \rightarrow \mathcal{P}(\Sigma)$ une fonction d'étiquetage

On dit qu'un mot $\omega := \omega_0 \dots$ est accepté ssi il existe une suite d'état $\sigma := \sigma_0 \dots$ tq $\forall i \in \mathbb{N}, \omega_i \in L(\sigma_i)$

Proposition 2 (GBA vers BA): Un ensemble multiple d'état acceptant peut être traduit en un ensemble en construisant un automate par une "counting construction". C'est à dire si $A = (S, I, \{F_1, \dots, F_n\}, \Sigma, T)$, alors il est équivalent à $A' = (S', I', F, \Sigma, T')$ telle que

- $Q' = Q \times \{1, \dots, n\}$
- $I' = I \times \{1\}$
- $F' = F_1 \times \{1\}$
- $\Delta' = \{((q, i), a, (q', j)) \mid (q, a, q') \in \Delta \text{ et si } q \in F_i, j = i + 1 \text{ mod } n \text{ sinon } j = i\}$

qui est bien un automate de Büchi (non généralisé) qui accepte les mêmes mots.

Preuve 2:

Proposition 3 (Intersection de BA): On définit l'automate reconnaissant l'intersection de deux langages comme $A' = (S', I', F', \Sigma, T')$ telle que

- $S' = S_1 \times S_2 \times \{1, 2\}$
- $T' = T'_1 \cup T'_2$
 - $T_1 = \{((q_1, q_2, 1), a, (q'_1, q'_2, i)), (q_1, a, q'_1) \in T_1, (q_2, a, q'_2) \in T_2, q_1 \in F_1 \Leftrightarrow i = 2, i \in [1, 2]\}$
 - $T_2 = \{((q_1, q_2, 2), a, (q'_1, q'_2, i)), (q_1, a, q'_1) \in T_1, (q_2, a, q'_2) \in T_2, q_2 \in F_2 \Leftrightarrow i = 1, i \in [1, 2]\}$
- $I' = I_1 \times I_2 \times \{1\}$
- $F' = \{(q_1, q_2, 2), q_2 \in F_2\}$

Il reconnaît l'intersection des langages reconnues par les automates de Büchi

Preuve 3:

1.3 La vérification de modèle

Definition 6: Techniques automatiques qui étant donné un modèle à nombre d'état fini d'un système et des propriétés formelles, vérifie systématiquement si ces propriétés restent vrai pour ce modèle.

On présente ici le déroulé de l'algorithme de vérification de modèle dans notre cas :

1. Création d'un automate A_m correspondant au cahier des charges d'une technologies, sous forme des différents états dans lequel la technologie sera.
2. Création d'un automate $A_{\neg\varphi}$ où φ représente les différentes contraintes que notre cahier des charges doit respecter, sous forme d'une formule logique en logique linéaire temporelle.
3. Création de l'automate Synchronisé $A_m \times A_{\neg\varphi}$
4. Si l'automate resultant est vide, alors c'est bon, sinon on renvoie un contre exemple.

Intéressons nous donc à la construction de A_m : Intuitivement, Le langage des A_m représente toutes les exécutions de M et celui de $A_{\neg\varphi}$ représente les exécutions ne satisfaisant pas la formule φ . Le langage du

produit est donc vide ssi tous les chemins de M satisfont φ . Sinon il existe un mot en commun, qui nous donne l'erreur.

En fait, l'automate est donné après transformation d'un système de transition $\mathcal{M}$ en un automate $A_{\mathcal{M}}$

2 Résolution

2.1 Création de A_m

2.2 Création de $A_{\neg\varphi}$

2.2.1 Structure de donnée

On présente ici l'algorithme de Gerth. Premièrement, on introduit une structure de donnée: On construit un graphe avec des noeuds décrit par:

- Nom: un string
- Incoming: Les arêtes qui arrivent sur le noeud représenté par le noeud voisin
- New: Un ensemble de formule LTL qui n'ont pas encore été considérées par l'algorithme
- Old: Un ensemble de formule LTL qui ont été considérées
- Next: Une formule LTL qui doit être vraie pour tout état dans les successeurs immédiats des états qui satisfont les propriétés de Old
- Father: Pendant la construction, les noeuds seront divisés, il contient le nom de l'autre noeud avec lequel il a été divisé

Remarque:

- On notera *Init* un noeud spécial qui désigne le noeud initial
- On note *Nodes_Set* une liste de noeud dont la construction est faite
- Father n'est pas nécessaire pour l'algorithme, c'est juste pour simplifier les raisonnements et la preuve de correction

2.2.2 Algorithm

On va supposer sans perte de généralité plusieurs hypothèses sur notre formule:

- L'opérateur X n'est pas utilisé, nous verrons plus tard comment lever cette restriction
- Toutes les négations sont poussées sur les variables.

Ainsi, l'opérateur V est en fait défini pour éviter une explosion exponentielle de la taille de la formule.

2.3 Création de $A_m \times A_{\neg\varphi}$

2.4 Conclusion et Commentaires

3 Correction

Proposition 4 (Correction de la création de $A_{\neg\varphi}$): L'algorithme de Gerth-Valdi-Peled-Wolper est correct

Preuve 4:

Proposition 5 (Correction de l'algorithme): Notre processus entier de vérification de modèle est correct.

Preuve 5:

Algorithm 2 Algorithmme de Gerth - Expand

Require: $Node, Nodes_Set$

```

if  $Node.New = \emptyset$  then
  if  $\exists ND \in Nodes\_Set | (ND.Old = Node.Old \wedge ND.Next = Node.Next)$  then
     $ND.Incoming \leftarrow ND.Incoming \cup Node.Incoming$ 
    return  $Nodes\_Set$ 
  end if
   $n \leftarrow new\_name()$ 
  return  $expand([Na:n, F:n, I:\{Node.Name\}, New:Node.Next, O:\emptyset, Nex:\emptyset], \{Node\} \cup Nodes\_Set)$ 
end if
 $\eta \in New$ 
 $Node.New := Node.new \setminus \{\eta\}$ 
if  $\eta \in \{F, T, VarProp, \neg VarProp\}$  then
  if  $(\eta = F) \vee (Neg(\eta) \in Node.Old)$  then
    return  $Nodes\_Set$ 
  end if
   $Node.Old := Node.Old \cup \{\eta\}$ 
  return  $expand(Node, Nodes\_Set)$ 
end if
if  $(\eta = \varphi \mathbf{U} \psi) \vee (\eta = \varphi \mathbf{V} \psi) \vee (\eta = \varphi \vee \psi)$  then
   $Node1 := [Na:new\_name(), F:Node.Name, I:Node.Incoming, New:Node.New \cup New1(\eta) \setminus Node.Old,$ 
 $O:Node.Old \cup \{\eta\}, Nex:Node.Next \cup Next1(\eta)]$ 
   $Node2 := [Na:new\_name(), F:Node.Name, I:Node.Incoming, New:Node.New \cup New2(\eta) \setminus Node.Old,$ 
 $O:Node.Old \cup \{\eta\}, Nex:Node.Next]$ 
  return  $expand(Node2, expand(Node1, Nodes\_Set))$ 
end if
if  $\eta = \varphi \wedge \psi$  then
  return  $expand([Na:Node.Name, F:Node.Father, I:Node.Incoming, New:Node.New \cup \{\varphi, \psi\} \setminus Node.Old,$ 
 $O:Node.Old \cup \{\eta\}, Nex:Node.Next], Nodes\_Set)$ 
end if
if  $\eta = X\varphi$  then
  return  $expand([Na:Node.Name, F:Node.Father, I:Node.Incoming, New:Node.New, O:Node.Old \cup \{\eta\},$ 
 $Nex:Node.Next \cup \{\varphi\}], Nodes\_Set)$ 
end if

```

Algorithm 3 Algorithmme de Gerth - Create Graph

Require: φ

```

 $n \leftarrow new\_name()$ 
return  $expand([Name:n, Father:n, Incoming:\{Init\}, New:\{\varphi\}, Old:\emptyset, Next:\emptyset], \emptyset)$ 

```

4 S'il y a pas assez de matière: Améliorations possibles

5 Bibliographie

1. Rob Gerth, Doron Peled, Moshe Y. Vardi, Pierre Wolper ; Simple On-the-fly Automatic Verification of Linear Temporal Logic ; <https://orbi.uliege.be/bitstream/2268/116644/1/GPVW95-pstv.pdf>
2. Christel Baier, Joost-Pieter Katoen ; Principles of Model Checking ; The MIT Press - ISBN: 9780262026499